Module 02 - Report

Stefano Vitale - ID: 654693

Integration of the partitioning function

The partitioning function provided in the reference code has been replaced with the **Murmur3 finalizer (fmix32)** developed in Module 1, which guarantees uniform key distribution even in the presence of low-entropy inputs.

Baseline Validation and Correctness

To ensure a rigorous and fair evaluation, the parallel implementation was validated against two distinct sequential baselines: the original reference code provided (`seq_original`, utilizing `std::unordered_map`) and an optimized sequential version (`seq`, utilizing the custom `SimpleHashTable`). Tests via a naive verifier (`naive_check.sh`) confirmed that all three implementations yield perfectly identical join counts and checksums across the same dataset. After this preliminary check of correctness, all executions reported below compare the best sequential version against the best parallelized version (**hashjoin_par.cpp vs hashjoin_seq.cpp**).

1 Optimization Strategy (hashjoin_par.cpp)

Histogram Computation This phase presents a dual memory access pattern: reading from `rel` is *sequential* (cache miss only at the start and every 8 records), while writing to `hist` is *random*, as target buckets depend on hash values in $[0, P-1]$. Parallelization follows a divide-and-conquer strategy to avoid locks and race conditions: the input is split into equal, non-overlapping chunks, one per thread, each of which computes a private `local_hist[t]` independently and lock-free. Once all threads join, the main thread performs a sequential reduction, summing the private histograms into the final global `hist`.

Exclusive Prefix Sum This phase computes the starting offset of each partition from the histogram via a simple loop with a strict loop-carried dependency: each step p depends on $p-1$. Given the small size of the histogram array (P = 2048), the sequential scan completes in a fraction of a millisecond, making it the optimal choice.

Scatter Phase (NUMA-Aware & Lock-Free) To eliminate synchronization overhead, the main thread pre-calculates a matrix of private write cursors (`local_next[t][pid]`) combining the global prefix sum with their local histograms. The actual scatter operation is then executed by spawning a fixed pool of T threads, which utilize these pre-assigned indices to move the records. This guarantees that each thread processes its input chunk and writes to strictly non-overlapping regions of the shared output array, making the phase completely lock-free. Furthermore, the output buffer is intentionally allocated via **std::malloc rather than a standard std::vector**. This prevents sequential zero-initialization by the OS, allowing the parallel scatter operations to trigger the Linux First-Touch policy and optimally distribute memory pages across the physical NUMA nodes.

Join One Partition The standard `std::unordered_map` is replaced with a custom `SimpleHashTable` using open addressing with linear probing, which eliminates chaining overhead and, since partition sizes are known from the histogram, allows exact pre-allocation of the table, removing all mid-loop reallocations.

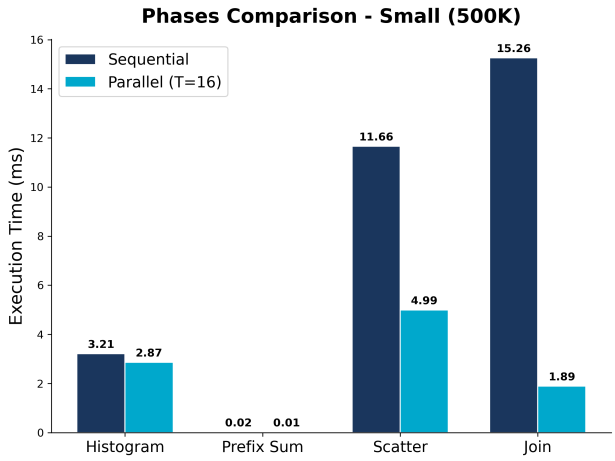
Complete Join Phase (Dynamic Load Balancing & Local Reduction) To prevent load imbalance caused by varying partition sizes, the workload is distributed dynamically using an atomic counter (`next_pid`) that acts as a lock-free task queue. Threads continuously fetch and process available partitions, accumulating

match counts and checksums into private, thread-local variables to completely eliminate synchronization overhead and race conditions. Following the parallel execution barrier (`join()`), the main thread performs a final sequential reduction of these private results to yield the global output.

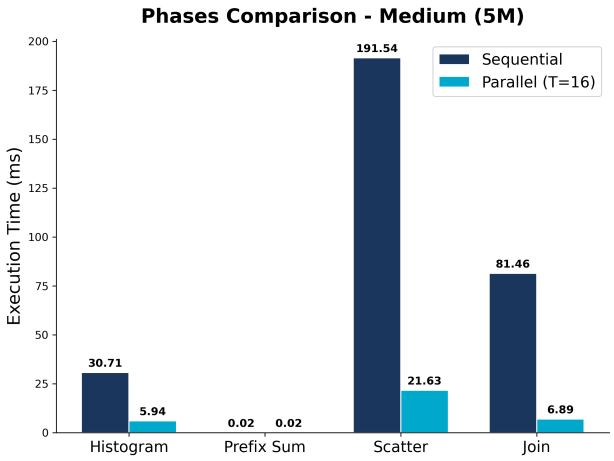
Failed Optimization Storing the Murmur3 hash values computed during the Histogram phase for reuse in the Scatter phase was evaluated, but empirical profiling showed a performance degradation. Since the algorithm is strictly memory-bound, the CPU cost of recomputing the hash on-the-fly is negligible compared to the overhead of transferring hundreds of megabytes of precomputed hashes from memory. On-the-fly recomputation was therefore kept as the most efficient strategy.

2 Metrics Analysis

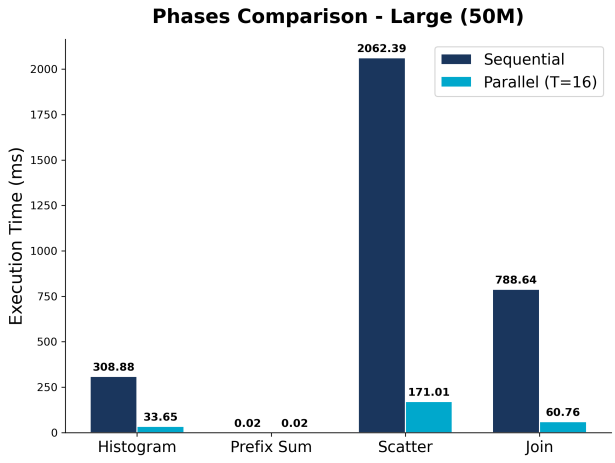
2.1 Breakdown and Full Execution Time



(a) Small (500K)



(b) Medium (5M)



(c) Large (50M)

Table 1: Performance Summary ($T = 16$, $P = 2048$).

Input	Seq (s)	Par (s)	Speedup	Eff (%)
500K	0.030	0.010	2.88x	18.0%
5M	0.304	0.035	8.51x	53.2%
50M	3.162	0.268	11.77x	73.5%

Figure 2: Execution time breakdown and performance metrics across different problem sizes.

Breakdown Execution Time Analysis

As shown in the figures and summarized in the table above, the parallel model ($T = 16$, $N = 50M$) achieves a robust overall speedup of 11.77x (73.5% efficiency). The phase breakdown reveals the non-uniform scaling behavior of the algorithm:

- **Join Phase:** This phase exhibits excellent scalability, dropping from 788.6 ms to 60.7 ms (∼13x speedup). Since both versions use the `SimpleHashTable`, the gain comes purely from parallelization, not from a data structure advantage.
- **Scatter Phase:** This phase is the primary bottleneck, consuming 171.0 ms (63% of the total parallel runtime). Despite being completely lock-free, 16 threads performing concurrent pseudo-random writes across 2048 partitions saturate the main memory bandwidth, hitting the *memory wall*.
- **Prefix Sum:** Execution time is identical in both versions (∼0.02 ms), as both rely on the same sequential implementation.
- **Small Workload Overhead (500K):** As evident in Table 1, the 500K dataset achieves a poor efficiency of 18.0% at $T = 16$. This occurs because the workload per thread is too small (∼31K records), causing the thread creation and synchronization overhead to dominate the actual computation. Optimal efficiency for this problem size requires scaling down the thread count.

2.2 Optimal Number of Partitions (P)

A parametric sweep was conducted to determine the impact of the number of partitions (P) on the total execution time. As shown in Table 2, the performance remains stable for a wide range of values ($2^9 \leq P \leq 2^{12}$), with an empirical minimum observed at $P = 2048$.

Partitions (P)	**Avg Records/Part.**	**Time ($T = 16$)**	**Time ($T = 32$)**
512	97,656	0.312030	0.286933
1024	48,828	0.271535	0.239760
2048	**24,414**	**0.270773**	**0.220195**
4096	12,207	0.287262	0.225448
8192	6,103	0.317226	0.245233
16384	3,051	0.348115	0.337839
32768	1,525	0.507848	0.511954

Table 2: Impact of partitions P on execution time comparing **physical cores** ($ThreadNum = 16$) and **logical cores** ($ThreadNum = 32$).

Analysis of Hardware Bounds As shown in Table 2, performance remains highly stable across a plateau ($1024 \leq P \leq 8192$). While $P = 2048$ emerges as the empirical minimum, this behavior can be explained by two distinct hardware bottlenecks:

- **Lower Bound (Cache Capacity):** Within the plateau (e.g., $P = 2048$), partitions are small enough (∼195 KB) to fit entirely within the L2 cache of a modern CPU core. Below $P = 1024$, partitions rapidly exceed L2 capacity, triggering costly L3 cache fallbacks that inevitably degrade performance.
- **Upper Bound (TLB Capacity):** During the parallel Scatter phase, threads write concurrently to P distinct memory regions. Pushing $P \geq 16384$ exceeds the Translation Lookaside Buffer (TLB) capacity, causing massive page table walks (TLB thrashing). This bottleneck is critical: at $P = 32768$, performance degrades severely, and doubling the threads ($T = 32$) yields zero benefit (0.507s vs 0.511s), as logical cores simply stall waiting for the same resources.

Possible Optimizations: Dynamic Partition Sizing Rather than static tuning, a production system could derive P at runtime directly from the hardware specifications with formulas such as the following one (applied per relation):

$$P \approx \frac{N \times \texttt{sizeof(record)}}{\text{L2 Cache Size}}$$

2.3 Strong Scaling Analysis

In the strong scaling evaluation, the problem size is fixed at $N = 50.000.000$ records per relation while varying the number of processing threads from 1 to 64. The goal is to measure the absolute speedup $S(p) = T_{seq}/T_{par}(p)$ and the corresponding efficiency $E(p) = S(p)/p$.

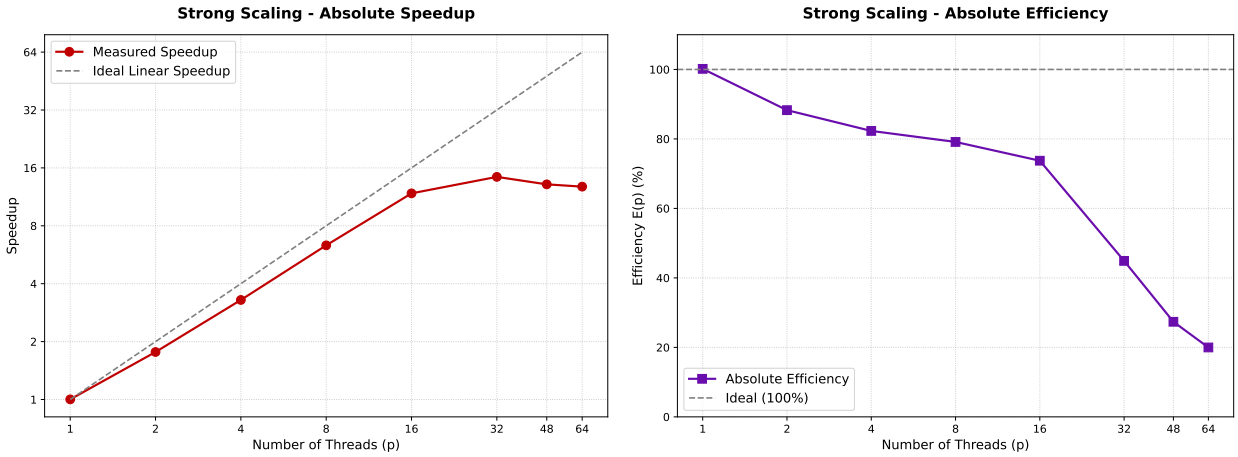


Figure 3: Strong Scaling evaluation. Left: Absolute Speedup (Log-Log scale) compared to the ideal linear boundary. Right: Absolute Efficiency (Log-Linear scale).

As shown in Figure 3, the algorithm scales nearly ideally up to $p = 8$, maintaining an efficiency above 79%. At $p = 16$, the system achieves an 11.7x speedup with a 73.5% efficiency. This represents the optimal configuration, as it perfectly maps the 16 physical cores available on the dual-socket NUMA node, avoiding contention on shared execution units.

The **absolute maximum speedup** is achieved at $p = 32$ (14.4×, with $T_{par} = 0.219$ s). However, this marginal gain in completion time comes at a **severe cost to efficiency**, which drops to 45.0%.

2.4 Weak Scaling Analysis

For this benchmark, a fixed workload of 5 million records per thread was assigned, scaling the total problem size from 5×10^6 ($p = 1$) up to 3.2×10^8 ($p = 64$). The Weak-Scaling Efficiency (WSE) is calculated as $WSE(p) = T(PS(1), 1)/T(PS(p), p)$.

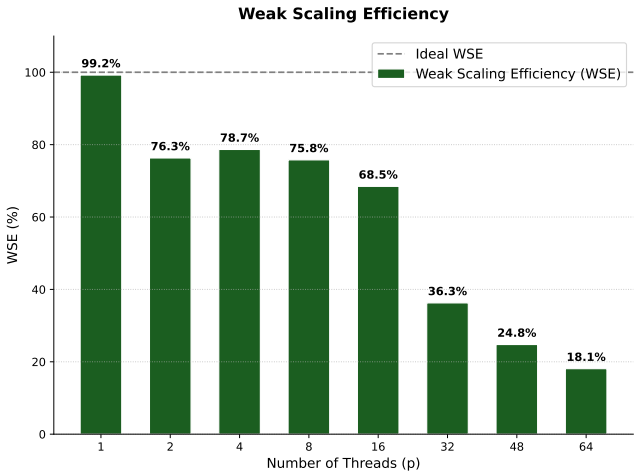


Figure 4: Weak-Scaling Efficiency (WSE) maintaining a constant workload of 5 million records per thread.

Gustafson's Law assumes that as the problem size scales proportionally with the processors, the parallelizable portion grows linearly while the serial portion remains constant, theoretically allowing efficiency to remain high. Figure 4 shows an initial drop from 99.2% to 76.3%. This architectural "tax" is caused by spawning and joining

threads five separate times per execution to preserve the phase-by-phase structure of the sequential baseline. Beyond this initial penalty, the implementation adheres well to Gustafson's model up to $p = 8$, stabilizing the WSE between 75% and 78%. Beyond p = 16, the WSE collapses sharply (68.5% $\rightarrow$ 36.3%) as threads exceed the physical core count, further saturating the already memory-bound scatter bottleneck.

2.5 Analysis of Problem Size (N)

To evaluate the robustness of the implementation, a parametric sweep was performed on the problem size N, ranging from 1 million to 150 million records per relation. For this test, the number of threads and partitions were kept constant at $T = 16$ and $P = 2048$. The results, including the total execution time and the system throughput, are reported in Table 3.

Data Scalability Analysis The throughput shows three distinct behaviors. At $N = 1$M, performance is low (148 M/s) because parallelization overhead dominates the small workload. Between $N = 25$M and $N = 125$M, the system reaches a stable plateau around 360 M/s, confirming that the algorithm scales linearly and effectively amortizes startup costs. The drop at $N = 150$M (329 M/s) is likely due to increasing pressure on the memory hierarchy and TLB as the total data grows beyond the optimal working set.

N	Time (s)	Thr. (M/s)
1M	0.0135	148.17
10M	0.0696	287.52
25M	0.1379	362.51
50M	**0.2667**	**375.01**
75M	0.4223	355.18
100M	0.5515	362.63
125M	0.6942	360.11
150M	0.9108	329.36

Table 3: Throughput and time vs sweeping data size N ($T = 16$, $P = 2048$).

3 Conclusion and Stress Test

The architectural breakdown highlighted that the combination of NUMA-aware memory allocation (First-Touch policy), lock-free data structures, and cache-sized partitioning ($P = 2048$) successfully mitigated the main memory bottleneck, achieving a $\sim$13x sub-linear speedup in the core Join phase. Furthermore, the scalability analysis demonstrated that maintaining a thread count strictly tied to the physical cores ($T \leq 16$) is crucial for high efficiency. Exceeding this threshold forces the use of logical cores, causing a severe efficiency drop.

Extreme Stress Testing and Final Remarks Ultimately, the Partitioned Hash Join is intrinsically a memory-bound problem. For this reason, it cannot achieve the near-perfect linear scaling typical of purely compute-bound algorithms. Despite this fundamental hardware constraint, the proposed implementation exhibits high stability and performance under extreme workloads.

Records per Relation (N)	Sequential Time (s)	Parallel Time ($T = 16$) (s)	Absolute Speedup
500 Million	38.465	4.145	9.27x
1 Billion	80.749	8.064	10.01x

Table 4: Extreme Stress Test: **Sequential vs. Parallel execution on massive datasets** ($P = 2048$).

As shown in Table 4, the system was subjected to massive datasets far exceeding typical L3 cache capacities. Processing half a billion records per relation ($\sim$8 GB of total raw payload) required nearly 38.5 seconds sequentially, but was completed in just 4.14 seconds by the parallel implementation. Scaling further to 1 billion records per relation ($\sim$16 GB of total raw payload), the sequential execution exceeded 80 seconds, while the parallel version reduced the execution time to 8.06 seconds, maintaining a solid 10x speedup.